Device Purchase Contract Management System — Complete Project Report

Project name: Device Purchase Contract Management System

Repository path: `purchase_dashboard/purchase_dashboard/`

Report date: June 8, 2026

Version: 0.0.0 (MVP+)

---

1. Executive Summary

This is an internal business web application for shops that buy used electronics and offer repair services. It digitizes the full workflow for:

- **Device purchase contracts** — customer data, device details, photos, signatures, PDF generation, and search
- **Repair orders** — intake, device/repair details, PDF receipts, status tracking
- **Invoices** — manual or repair-order-based billing with VAT line items and PDF output

The system is a **monorepo** with a React frontend and an Express/Prisma/PostgreSQL backend. It supports **English and German**, JWT authentication, per-user data isolation, and local file storage for uploads and PDFs.

The original PRD focused on purchase contracts only. Repair orders and invoices were added as extensions beyond the original “out of scope” list, while accounting, inventory, and multi-shop support remain out of scope.

---

2. Problem Statement

Shops buying used devices often rely on paper contracts, scattered photos, and manual notes. That leads to:

- Slow contract creation
- Lost or incomplete records
- Difficulty finding past transactions
- Weak proof of ownership and device condition
- Dispute risk around stolen devices or account locks

This system centralizes purchase contracts, repair intake, and invoicing in one dashboard.

---

3. Goals & Scope

In scope (implemented)

Area	Status
User login / signup	✅
Purchase contract wizard (draft → complete)	✅
Photo & signature uploads	✅
PDF contract generation	✅
Contract search & list	✅
Dashboard with today’s metrics	✅
Shop settings (logo, branding)	✅
Repair orders (CRUD, PDF, search)	✅
Invoices (CRUD, VAT, PDF, RO link)	✅
EN / DE internationalization	✅

Out of scope (by design)

- AI chatbot
- Inventory / stock tracking
- Accounting / export modules
- Multi-shop / multi-tenant
- WhatsApp integration
- Advanced reporting / analytics
- Complex role-based permissions
- Customer-facing portal

---

4. Technology Stack

Frontend

Technology	Purpose
React 19	UI framework
Vite 8	Dev server & build
TypeScript 6	Type safety
Tailwind CSS 3	Styling
React Router 7	Routing
React Hook Form	Settings form
react-signature-canvas	Digital signatures
lucide-react	Icons

> **Default dev URL:** `http://localhost:5173`

Backend

Technology	Purpose
Node.js + Express 5	REST API
TypeScript 5	Type safety
Prisma 6	ORM
PostgreSQL	Database
Zod	Request validation
JWT + bcrypt	Auth
Multer	File uploads
PDFKit	PDF generation

> **Default API URL:** `http://localhost:4000`

Infrastructure

- PostgreSQL (Docker or pgAdmin-managed; commonly port `5433` in dev)
- Local disk storage under `backend/storage/`

---

5. Architecture Overview

```
flowchart TB
    subgraph Frontend["Frontend (React + Vite)"]
        Pages[Pages / Components]
        API[API Client]
        i18n[Language Provider]
        Auth[Auth Context]
    end

    subgraph Backend["Backend (Express)"]
        Routes[Routes]
        Controllers[Controllers]
        Services[Services]
        Validators[Zod Validators]
        PDF[PDF Service]
        Files[File Service]
    end

    subgraph Data["Persistence"]
        DB[(PostgreSQL)]
        Storage[Local Storage]
    end

    Pages --> API
    API -->|JWT REST| Routes
    Routes --> Controllers
    Controllers --> Services
    Services --> DB
    Services --> PDF
    Services --> Storage
    PDF --> Storage
```

Design patterns

- **Layered backend:** Routes → Controllers → Services → Prisma
- **Per-user isolation:** All records scoped by `userId`
- **Draft → Complete workflow** for contracts
- **Auto-numbering** for contracts, repair orders, and invoices
- **Relative storage paths** in DB; absolute paths resolved at runtime

---

6. Project Structure

```
purchase_dashboard/
├── package.json              # Root scripts (dev:frontend, dev:backend)
├── README.md
├── device_purchase_contract_prd.md   # Original PRD
├── backend/
│   ├── prisma/
│   │   ├── schema.prisma
│   │   └── migrations/
│   ├── src/
│   │   ├── app.ts
│   │   ├── server.ts
│   │   ├── config/
│   │   ├── controllers/
│   │   ├── middlewares/
│   │   ├── routes/
│   │   ├── services/
│   │   ├── utils/
│   │   └── validators/
│   ├── storage/              # Generated at runtime
│   │   ├── contracts/
│   │   ├── repair-orders/
│   │   └── invoices/
│   └── .env
└── frontend/
    ├── public/
    │   └── company_logo.png
    ├── src/
    │   ├── api/
    │   ├── app/layout/       # Sidebar, Topbar, AppLayout
    │   ├── auth/
    │   ├── components/
    │   ├── hooks/
    │   ├── i18n/
    │   ├── pages/
    │   ├── services/
    │   ├── types/
    │   └── utils/
    └── .env
```

---

7. Database Schema

Models

Model	Description
`User`	Staff accounts (email, password hash)
`ShopSettings`	Per-user shop branding (name, address, logo)
`Contract`	Device purchase contracts
`ContractFile`	Uploaded photos per contract
`RepairOrder`	Repair intake records
`Invoice`	Billing documents
`InvoiceItem`	Line items with VAT

Key enums

- **ContractStatus:** `Draft`, `Completed`, `Cancelled`
- **RepairOrderStatus:** `Received`, `InProgress`, `WaitingForParts`, `ReadyForPickup`, `Completed`, `Cancelled`
- **InvoicePaymentMethod:** `Cash`, `BankTransfer`, `Card`, `Other`
- **InvoicePaymentStatus:** `Paid`, `Open`, `Cancelled`

Document numbering

Entity	Format	Example
Contract	`{YYYY}-{seq}`	`2026-0001`
Repair Order	`RO-{YYYY}-{seq}`	`RO-2026-0001`
Invoice	`INV-{YYYY}-{seq}`	`INV-2026-0001`

All numbers are unique per user.

---

8. Module Breakdown

8.1 Authentication

Status: ✅ Complete

- Open signup and login
- JWT bearer tokens stored in `localStorage`
- Protected routes via `ProtectedRoute`
- Endpoints: `POST /api/auth/signup`, `POST /api/auth/login`, `GET /api/auth/me`

8.2 Purchase Contracts

Status: ✅ Complete (core PRD module)

Features

- Multi-step **Contract Wizard** (customer → device → confirmations → photos → signature → review)
- Draft save and resume
- Required uploads: ID front, device front/back, IMEI photo, customer signature
- Ownership and lock-removal confirmations
- IMEI/serial validation
- Auto contract number generation
- PDF generation on completion
- Contract list, detail view, cancel/delete
- Dedicated search page (`/contracts/search`)
- Embedded compact wizard on dashboard

Contract completion rules

- Full customer and device data
- Positive purchase price
- At least IMEI or serial number
- All ownership/lock checkboxes `true`
- Required file uploads and signature

Storage Layout

```
storage/contracts/{userId}/{contractNumber}/
  ├── contract.pdf
  ├── signature.png
  └── [uploaded images]
```

8.3 Dashboard

Status: ✅ Complete

Stat cards

Card	Source
Contracts today	Count of contracts completed today (`updatedAt`)
Total purchase value today	Sum of `purchasePrice` for today's completed contracts
Current contracts	All non-cancelled contracts
Draft contracts	Draft count

Other dashboard elements

- Recent contracts table (last 10)
- Quick actions card
- Embedded contract wizard with auto-refresh on completion

> **Recent fix:** `todayPurchaseTotal` now correctly sums real DB values (Prisma `Decimal` converted to numbers); dashboard refreshes after completing a contract from the wizard.

8.4 Repair Orders

Status: ✅ Complete

Features

- Create / edit repair orders
- Auto-generated `repairOrderNumber` (not client-overridable)
- Customer, device (preset dropdowns + custom "Other"), and repair sections
- Multi-select accessories (checkboxes)
- Field validation with inline errors and success toasts
- Status workflow (6 statuses)
- Inline search + status filter on list page (250ms debounce)
- PDF auto-generation on save
- Post-save actions: View, Edit, Download PDF, Create Invoice
- Linked invoice display and duplicate-invoice prevention

Storage Layout

```
storage/repair-orders/{userId}/{repairOrderNumber}/repair-order.pdf
```

8.5 Invoices

Status: ✅ Complete

Features

- Manual invoice creation (`/invoices/new`)
- Create from repair order (copies customer, device, repair info)
- Editable invoice number and date (auto-generated if blank)
- Payment method and status
- Unlimited line items: Description, Qty, Unit Price, VAT %, Total
- VAT presets: **0%, 10%, 13%, 20%, Other/custom**
- Auto-calculated net, VAT, gross totals
- Optional manual total overrides (effective totals shown in summary)
- Customer name validation before save
- PDF generate and download
- `pdfPath` cleared on edit (stale PDF prevention + warning banner)
- Error toasts for PDF generation and delete failures
- Invoice ↔ Repair Order linking (visible on both sides)
- One invoice per repair order (UI guard + backend 409)
- Inline list search by invoice #, customer, phone, and date

Storage Layout

```
storage/invoices/{userId}/{invoiceNumber}/invoice.pdf
```

Known minor limitations

- PDF table renders max **14 line items** visually (totals include all items)
- Currency symbol hardcoded as `$` in PDFs
- Search capped at 100 results (no pagination)

8.6 Settings

Status: ✅ Complete

- Shop name, address, phone, email, owner name
- Logo upload (base64, max 2 MB)
- Used in contract, repair order, and invoice PDF headers
- Persisted per user in `ShopSettings`

8.7 Internationalization (i18n)

Status: ✅ Complete

- Languages: **English (`en`)** and **German (`de`)**
- `LanguageProvider` with `useLanguage()` hook
- Localized: navigation, pages, forms, tables, errors, toasts, payment labels
- Money and date formatting helpers

---

9. API Reference

All routes except `/health` and `/api/auth/*` require `Authorization: Bearer <token>`.

Auth

Method	Endpoint	Description
POST	`/api/auth/signup`	Register
POST	`/api/auth/login`	Login
GET	`/api/auth/me`	Current user

Contracts

Method	Endpoint	Description
GET	`/api/contracts/search`	Search contracts
GET	`/api/contracts/validate-identifiers`	IMEI/serial check
POST	`/api/contracts/draft`	Create draft
PATCH	`/api/contracts/:id/draft`	Update draft
GET	`/api/contracts/:id`	Get contract
POST	`/api/contracts/:id/files`	Upload file
POST	`/api/contracts/:id/signature`	Upload signature
POST	`/api/contracts/:id/complete`	Complete contract
POST/DELETE	`/api/contracts/:id/cancel`	Cancel contract
GET	`/api/contracts/:id/pdf`	View PDF

Method	Endpoint	Description
GET	`/api/contracts/:id/pdf/download`	Download PDF

Dashboard

Method	Endpoint	Description
GET	`/api/dashboard`	Summary + recent contracts

Repair Orders

Method	Endpoint	Description
GET	`/api/repair-orders/search`	Search/list
POST	`/api/repair-orders/`	Create
GET	`/api/repair-orders/:id`	Get (includes linked invoices)
PATCH	`/api/repair-orders/:id`	Update
PATCH	`/api/repair-orders/:id/status`	Update status
POST	`/api/repair-orders/:id/pdf`	Generate PDF
GET	`/api/repair-orders/:id/pdf`	View PDF
DELETE	`/api/repair-orders/:id`	Delete

Invoices

Method	Endpoint	Description
GET	`/api/invoices/search`	Search/list
POST	`/api/invoices/`	Create
POST	`/api/invoices/from-repair-order/:id`	Create from RO
GET	`/api/invoices/:id`	Get
PATCH	`/api/invoices/:id`	Update (clears `pdfPath`)
PATCH	`/api/invoices/:id/status`	Payment status only
POST	`/api/invoices/:id/pdf`	Generate PDF
GET	`/api/invoices/:id/pdf`	View PDF
DELETE	`/api/invoices/:id`	Delete

Settings

Method	Endpoint	Description
GET	`/api/settings`	Get shop settings
PUT	`/api/settings`	Save shop settings

---

10. Frontend Routes

Route	Page	Description
`/login`	LoginPage	Authentication
`/dashboard`	DashboardPage	Summary + wizard
`/contracts/new`	NewContractPage	Full contract wizard
`/contracts`	ContractsPage	Contract list
`/contracts/search`	SearchContractsPage	Advanced search
`/contracts/:id`	ContractDetailPage	View/edit contract
`/repair-orders/new`	NewRepairOrderPage	New repair order
`/repair-orders`	RepairOrdersPage	RO list + search
`/repair-orders/:id`	RepairOrderDetailPage	RO detail + actions
`/invoices/new`	NewInvoicePage	New invoice
`/invoices`	InvoicesPage	Invoice list + search
`/invoices/:id`	InvoiceDetailPage	Invoice detail
`/settings`	SettingsPage	Shop branding

---

11. UI / Branding

- Primary color: `#1B48AC`
- Company logo: `frontend/public/company_logo.png`
- Login page: blue header with logo, white form body
- Sidebar: dark blue with logo (`mix-blend-screen` on logo)
- Responsive layout with mobile sidebar drawer
- Shared components: `ConfirmDialog`, `AppToast`, `StatCard`, `StatusBadge`

---

12. Security

Measure	Implementation
Authentication	JWT (configurable `JWT_SECRET`)
Password storage	bcrypt hashing
Authorization	All data scoped to authenticated `userId`
CORS	Configurable `CORS_ORIGINS`
Upload limits	`MAX_UPLOAD_SIZE_MB` (default 5 MB)
Validation	Zod on all API inputs

> **Not implemented:** role-based access, audit logs, rate limiting, HTTPS enforcement (deployment concern).

---

13. Environment Configuration

Backend (`backend/.env`)

Variable	Purpose
`DATABASE_URL`	PostgreSQL connection string
`JWT_SECRET`	Token signing (min 16 chars)
`PORT`	API port (default 4000)
`HOST`	Bind address
`CORS_ORIGINS`	Allowed frontend origins
`SHOP_*`	Default shop info fallback
`MAX_UPLOAD_SIZE_MB`	Upload size limit

Frontend (`frontend/.env`)

Variable	Purpose
`VITE_API_BASE_URL`	Backend API URL (e.g. `http://localhost:4000`)

---

14. Setup & Run Instructions

Prerequisites

- Node.js 18+
- PostgreSQL
- npm

Backend Execution

```
cd backend
cp .env.example .env
npm install
npm run prisma:generate
npm run prisma:migrate
npm run dev
```

Frontend Execution

```
cd frontend
cp .env.example .env
npm install
npm run dev
```

Monorepo Quickstart (Root Folder)

```
npm run install:all
npm run dev:backend    # run in terminal 1
npm run dev:frontend   # run in terminal 2
```

---

15. Implementation Status Summary

Module	Completion	Notes
Auth	✅ 100%	Signup, login, JWT
Contracts	✅ 100%	Full PRD workflow
Dashboard	✅ 100%	Dynamic today totals fixed
Search (contracts)	✅ 100%	Dedicated page + list filters
Settings	✅ 100%	Logo + shop info
Repair Orders	✅ 100%	Beyond original PRD scope
Invoices	⚠ ~95%	Core complete; PDF 14-row display cap
i18n (EN/DE)	✅ 100%	All major pages
Multi-shop	❌ 0%	Out of scope
Accounting	❌ 0%	Out of scope
Inventory	❌ 0%	Out of scope

---

16. Known Limitations & Technical Debt

1. **No pagination** on search endpoints (hard cap ~100 results).
2. **Invoice PDF** shows only first 14 rows in the table; VAT breakdown uses all rows.
3. **Currency** hardcoded as `$` in PDFs regardless of locale.
4. **Single-user shop model** — no multi-location or staff roles.
5. **Local file storage only** — no S3/cloud storage.
6. **Contract search page** still exists separately; repair orders and invoices use inline list search only.
7. **Limited automated tests** — mostly manual/API scripts in `backend/scripts/`.
8. **Repair order delete** sets invoice `repairOrderId` to null (invoice kept, link lost).

---

17. Recent Development Highlights

Work completed during the latest development cycle:

- Dashboard **Total Purchase Value Today** fixed (dynamic DB sum, decimal parsing, refresh on contract complete).
- Full **English/German** translations for invoices and repair orders.
- **Repair order form** upgraded (preset dropdowns, validation, auto PDF, post-save actions).
- **Invoice module** gaps closed:
- Customer name validation.
- VAT preset dropdowns (0/10/13/20/custom).
- Stale PDF prevention on edit.
- Error toasts for PDF/delete flags.
- Effective totals with overrides.
- Invoice ↔ Repair Order linking and duplicate prevention rules.
- Login page and sidebar **branding** updated with company logo.
- Removed separate invoice/repair-order search pages (integrated inline search on list pages).

---

18. Conclusion

The **Device Purchase Contract Management System** is a functional internal operations platform. The original PRD scope for purchase contracts is fully implemented.

Repair orders and invoices extend the product into repair-shop workflows without adding accounting or inventory complexity.

The codebase is maintainable, typed end-to-end, and ready for production deployment with PostgreSQL and a standard Node.js hosting setup. Future enhancements could include pagination, cloud storage, locale-aware currency in PDFs, and optional role-based access — without changing the current simple shop workflow.

End of report.